

Classification de cellules sur images histologiques

Projet IA & Optimisation — ISEP Paris, 2026

Rapport de justification des choix techniques

Équipe : Yann Roquigny-Martin & Victor Poussier

Synthèse

Ce rapport accompagne le notebook **projet_IA_optimisation.ipynb**. Il justifie les choix de features et de modèles, détaille la conformité au cahier des charges, et décrit la structuration du code. L'objectif est de classer des patches d'images histologiques (colorées en H&E) parmi quatre types cellulaires : **Fibroblast**, **Lymphocyte**, **Plasma** et **Tumor**. Le jeu d'entraînement compte **400 images** parfaitement équilibrées (100 par classe). Notre meilleure approche, un **SVM à noyau RBF**, atteint **66,25 % d'accuracy** en validation croisée 5-fold, contre 62,0 % pour la Random Forest.

1. Exploration des données

L'exploration confirme un jeu de données **équilibré** (100 patches par classe), ce qui justifie l'usage de l'*accuracy* comme métrique principale et rend inutile tout rééquilibrage. L'affichage de patches représentatifs par classe révèle les signatures visuelles qui ont guidé l'ingénierie des features :

- **Lymphocyte** : petit noyau rond très sombre sur fond clair — forte circularité, faible surface.
- **Tumor** : texture dense et irrégulière, nombreuses cellules — surface élevée, contraste fort.
- **Plasma** : cellule allongée et sombre — aspect ratio élevé, intensité intra-noyau faible.
- **Fibroblast** : cellule fusiforme claire, tissu étalé — forme allongée, intensité moyenne élevée.

Des nuages de points (*area_ratio* vs *gray_mean*, *circularity* vs *norm_perimeter*, etc.) montrent un **recouvrement partiel** entre classes : la tâche est non triviale et justifie l'emploi de modèles non linéaires plutôt qu'un simple séparateur linéaire.

2. Choix et justification des features

Conformément à la consigne, **toutes les features sont « homemade »**, implémentées uniquement avec numpy. La segmentation noyau/fond repose sur un **seuillage d'Otsu recodé à la main** (maximisation de la variance inter-classes sur l'histogramme), sans recours à scipy ou OpenCV. En coloration H&E les cellules sont plus sombres que le fond, d'où un masque défini par *gris* < *seuil*. 26 features sont extraites, regroupées selon les trois familles exigées par le sujet :

Famille	Features	Justification
Aire	area_ratio, bb_area_ratio, extent	Sépare les petites cellules (lymphocytes) des amas denses (tumeurs).
Forme	circularity, aspect_ratio, norm_perimeter	Distingue les noyaux ronds des cellules allongées (plasma, fibroblaste).

Intensité	gray mean/std/contrast/skew, R/G/B mean+std, histogramme 8 bins, cell_mean/std (intra-masque)	Capture la densité de chromatine et les nuances de coloration H&E propres à chaque type.
------------------	---	--

L'ensemble des features est sauvegardé dans **features_train.csv** (recommandation du sujet), ce qui évite de recalculer la segmentation à chaque exécution et garantit la reproductibilité. La **même fonction d'extraction** est réutilisée pour le jeu de test, assurant une cohérence parfaite entre entraînement et prédiction.

3. Choix des frameworks de Machine Learning

Deux algorithmes vus en cours sont comparés, chacun intégré dans un **pipeline scikit-learn** identique : *StandardScaler* → *SelectKBest(ANOVA F)* → *classifieur*.

Pourquoi ce pipeline ?

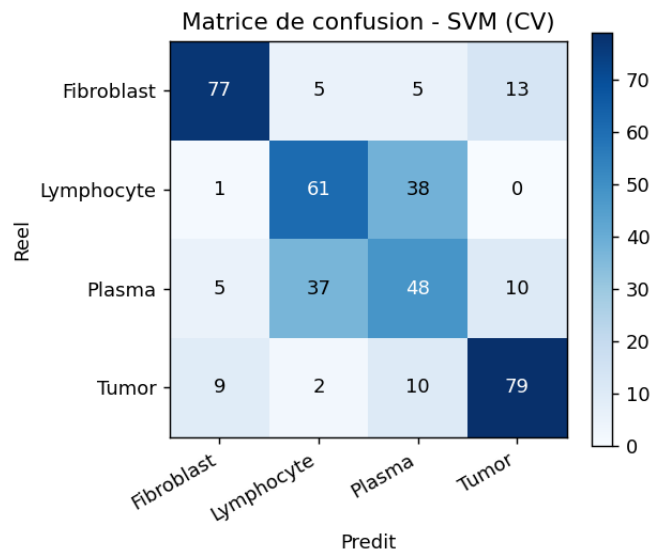
- **Normalisation (StandardScaler)** : indispensable pour le SVM, dont le noyau RBF repose sur des distances ; sans elle, les features à grande échelle (moyennes RGB ~150) écraseraient les ratios (~0–1).
- **Sélection (SelectKBest, test F d'ANOVA)** : retient les k features les plus discriminantes (k testé parmi 10, 15, 20, all) pour limiter le sur-apprentissage et le bruit.
- **Validation croisée stratifiée (StratifiedKFold, 5 plis)** : préserve la distribution des classes dans chaque pli et fournit une estimation robuste de la performance.

Algorithmes et hyperparamètres

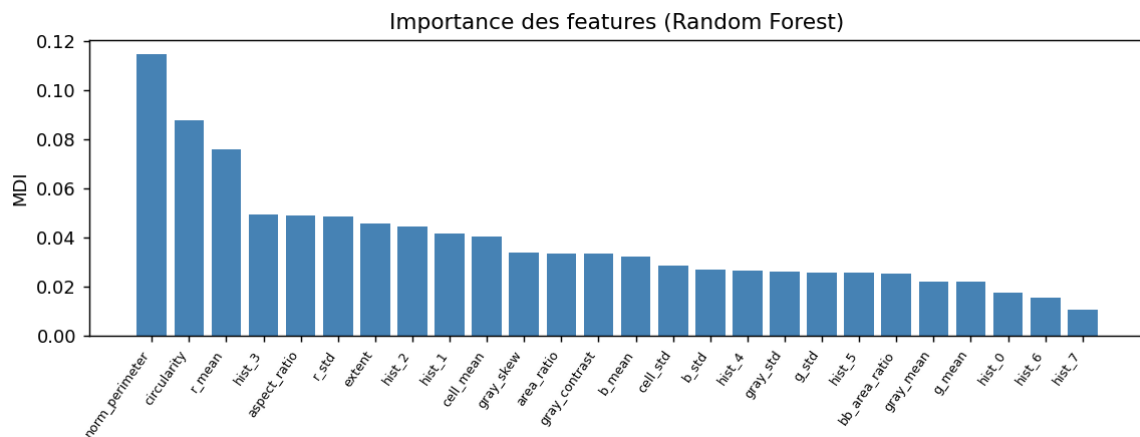
La recherche d'hyperparamètres utilise **GridSearchCV** (exploration exhaustive, scoring = accuracy) — c'est l'algorithme d'optimisation demandé par le sujet.

Modèle	Hyperparamètres explorés	Meilleure config	Acc. CV
SVM RBF	$C \in \{0.1, 1, 10, 100\}$; $\gamma \in \{\text{scale}, \text{auto}, 0.01, 0.1\}$; $k \in \{10, 15, 20, \text{all}\}$	$C=10$, $\gamma=0.01$, $k=\text{all}$	0,6625
Random Forest	$n_{\text{estimators}} \in \{100, 200, 300\}$; $\text{max_depth} \in \{\text{None}, 10, 20\}$; $\text{min_samples_split} \in \{2, 5\}$; $k \in \{10, 15, 20, \text{all}\}$	$n=300$, $\text{depth}=\text{None}$, $\text{split}=2$, $k=\text{all}$	0,6200

Le **SVM RBF est retenu** : il l'emporte de plus de 4 points sur la Random Forest et présente un écart-type inter-plis plus faible (0,034 contre 0,042), donc une meilleure stabilité. Le choix du modèle final est **automatisé** dans le code (comparaison des moyennes CV).



Matrice de confusion du SVM (prédictions croisées). Les confusions principales opposent Fibroblast et Plasma, visuellement proches.



Importance des features (Random Forest) : les statistiques d'intensité et de surface dominant.

4. Conformité aux instructions

Exigence du sujet	Statut	Où dans le notebook
Exploration : nb samples, distribution, apparence par classe	OK	Section I (cellules 3–4)
Features homemade (numpy) : aire, forme, intensité	OK	Section II (cellules 6–8)
Sauvegarde des features en CSV	OK	features_train.csv
Au moins 2 algorithmes de classification	OK	SVM + Random Forest
Sélection de features	OK	SelectKBest
Normalisation des features	OK	StandardScaler
Réglage des hyperparamètres	OK	GridSearchCV
Algorithme d'optimisation	OK	GridSearchCV (recherche exhaustive)
Validation croisée	OK	StratifiedKFold 5 plis
Code de la phase de test → test.csv	OK	Section IV (fonction predict_test)

Noms de l'équipe dans le fichier	OK	Cellule markdown d'introduction
Restart & Run All sans erreur	OK	Vérifié (15 cellules, exec 1–15)
Lignes ≤ 80 caractères	OK	0 ligne hors norme
Nom de fichier sans espace/caractère spécial	OK	projet_IA_optimisation.ipynb

5. Qualité et clarté du notebook

Le notebook **suit fidèlement le plan du sujet** (I Exploration, II Features, III Entraînement, IV Test). Chaque section est introduite par une **cellule markdown** qui justifie les choix (stratégie de segmentation, tableau des features, rôle de chaque étape du pipeline). Le code est **modulaire** : la logique d'extraction est factorisée en fonctions réutilisables (*to_grayscale*, *otsu_threshold*, *get_mask*, *area_features*, *shape_features*, *intensity_features*, *extract_features*), elles-mêmes appelées à l'entraînement comme au test. Une graine aléatoire fixe (*RANDOM_STATE* = 42) garantit la reproductibilité. Les résultats sont accompagnés de visualisations (distribution, scatter plots, comparaison des plis, matrice de confusion, importance des features) qui rendent l'analyse lisible et vérifiable.

Livrables : projet_IA_optimisation.ipynb (code) et test.csv (prédictions, généré lors de la phase de test).